8/7/2025

Internship Project Report

Data Science and Machine Learning

Project Title:

Autistic Spectrum Disorder Screening Data for Toddlers

Name : Sourab

Chapter 1:

Introduction

Autistic Spectrum Disorder (ASD) is a neuro developmental condition associated with significant healthcare costs, and early diagnosis can significantly reduce these. Unfortunately, waiting times for an ASD diagnosis are lengthy and procedures are not cost effective. The economic impact of autism and the increase in the number of ASD cases across the world reveals an urgent need for the development of easily implemented and effective screening methods. Therefore, a time-efficient and accessible ASD screening is imminent to help health professionals and inform individuals whether they should pursue formal clinical diagnosis. The rapid growth in the number of ASD cases worldwide necessitates datasets related to behaviour traits. However, such datasets are rare making it difficult to perform thorough analyses to improve the efficiency, sensitivity, specificity and predictive accuracy of the ASD screening process.

Presently, very limited autism datasets associated with clinical or screening are available and most of them are genetic in nature. Hence, we propose a new dataset related to autism screening of toddlers that contained influential features to be utilised for further analysis especially in determining autistic traits and improving the classification of ASD cases. In this dataset, we record ten behavioural features (Q-Chat-10) plus other individuals characteristics that have proved to be effective in detecting the ASD cases from controls in behaviour science.

Problem Statement: Early detection of Autism Spectrum Disorder(ASD) in toddlers is often delayed due to limited access to fast, affordable, and reliable screening methods. This project aims to develop a machine learning model using behavioral data from the Q-Chat-10 tool to enable accurate, early identification of ASD traits, supporting timely intervention and improved developmental outcomes.

Why this study is Important?

Studying autism is essential because it helps us better understand a complex neurodevelopmental disorder that significantly affects communication, behaviour, and social interaction, impacting individuals, families, and society as a whole. Research and awareness not only enable earlier and more accurate diagnosis—key to effective intervention and improved quality of life—but also help develop targeted treatments, reduce stigma, and inform public policy.

Research on autism sheds light on its causes and risk factors, many of which are still not fully understood, and can lead to breakthroughs that support early intervention strategies. Early detection is particularly critical, as interventions during early development can help mitigate

symptoms and enhance social, cognitive, and adaptive functioning throughout life. This is important given the variation in autism's presentation and the high rates of co-occurring conditions.

Broader autism study also benefits the understanding of other neurodevelopmental disorders, as insights from autism research can inform approaches to related conditions. Furthermore, public awareness and education help break down misconceptions, reduce stigma, and create more inclusive communities by recognizing the strengths and unique perspectives of autistic people.

Developing accurate and efficient autism detection models, including those using machine learning, is part of this important work—streamlining diagnosis and connecting individuals to vital resources much sooner than traditional methods often allow. Such research directly improves the lives of autistic people and their families, supporting their inclusion and success at every stage of life

Goals of this Project

The primary goals of an autism detection model project are to enable early, accurate, and efficient identification of individuals with Autism Spectrum Disorder(ASD), using advanced computational methods such as machine learning. These goals typically include:

- Early Detection: Facilitate earlier diagnosis of autism, ideally during infancy or early childhood, so that interventions can be initiated as soon as possible, leading to improved developmental outcomes.
- High Accuracy and Efficiency: Maximize the accuracy, precision, and generalizability of the model, ensuring that ASD detection is both reliable and applicable across diverse populations with minimized time and cost.
- Accessibility: Develop models that are accessible and usable for clinicians and potentially for non-specialist use, possibly through web-based tools or digital platforms.
- Minimally Invasive and Scalable Solutions: Leverage minimal and easily obtainable data (such as medical records, behavioural questionnaires, or brief screening tools), reducing reliance on lengthy and subjective assessments.
- Reducing Subjectivity and Bias: Improve upon traditional screening tools by reducing subjectivity, cultural bias, and interpretation errors that may affect diagnosis.
- Support for Targeted Intervention: Enable timely connection to interventions and resources, aiming to reduce the burden on families and improve quality of life for autistic individuals.
- Advancing Research and Understanding: Inform further research on autism, including understanding risk factors and heterogeneity, and supporting continuous refinement of detection methods.

In summary, the core goal is to create a model that allows practical, reliable, and early identification of ASD to support timely intervention, reduce societal burden, and advance scientific understanding

Data Collection

The dataset used for this analysis was collected from Kaggle, a reputable online platform for data science datasets and competitions. Kaggle provides access to large, curated collections of real-world data contributed by researchers and the wider data science community. This ensures that the data is both accessible and standardized for research use.

Source: The dataset, titled Autism_Screening_Data_Combined.csv, was downloaded directly from Kaggle.

Why Kaggle? Kaggle is widely recognized for hosting clean, ready-to-analyze datasets, often with rich metadata and community support. Using a Kaggle dataset ensures transparency and reproducibility for research.

Details Captured: The dataset contains responses to ten binary autism screening questions (A1–A10), as well as demographic details (Age, Sex), medical history (history of jaundice, family history of ASD), and the screening result (Class).

Chapter 2:

Dataset Preprocessing

Data Preprocessing is a very important step in any data analysis or ML project. It involves preparing the data and cleaning it to make sure that the dataset is ready for analysis or modelling. This step includes several sub-steps like Data Cleaning, Feature Engineering and Data Transformation. These sub-steps help in removing inconsistencies, modify the dataset structure, and ensure it aligns with the modelling requirements.

Data Cleaning

Data cleaning is the process of identifying missing values, outliers, unnecessary data or incorrect entries and rectifying these issues. It makes sure that the data is consistent and reliable before moving further with the analysis.

In this project, the dataset had some columns with missing values, which needed to be addressed. The cleaning process was carried out using the following steps:

Dropping Unnecessary Columns:

The first step involved removing unnecessary columns that did not contribute to the predictive analysis:

```
df2 = df2.drop(['Case_No','Qchat-10-Score','Ethnicity','Who completed the test'],axis=1)
```

Rationale for column removal:

Case_No: Sequential identifier with no predictive value

Qchat-10-Score: Redundant as it represents the sum of individual behavioral features (A1-A10)

Ethnicity: Removed to focus on behavioral patterns and avoid potential bias

Who completed the test: Administrative information not relevant to ASD trait prediction

Age Unit Conversion:

The age data was converted from months to years to provide more intuitive age representation

```
df2['Age_Mons'] = df2['Age_Mons'] / 12
```

Column Renaming:

Column names were standardized for better readability and consistency

```
df2.rename(columns={
    'Age_Mons': 'Age',
    'Family_mem_with_ASD': 'Family_ASD',
    'Class_ASD Traits': 'Is_ASD'
}, inplace=True)

df1.rename(columns={
    'Jauundice': 'Jaundice',
    'Class': 'Is_ASD'
}, inplace=True)
```

These changes improve code readability and maintain consistency across datasets.

Data Standardization:

String data in categorical columns was standardized to ensure consistency:

```
for col in ['Is_ASD', 'Jaundice', 'Family_ASD']:
    df2[col] = df2[col].str.strip().str.capitalize()
```

The strip() function removed leading and trailing whitespaces, while capitalize() ensured consistent formatting with the first letter capitalized and remaining letters in lowercase.

Missing Value Analysis:

A comprehensive check for missing values was performed:

```
df2.isnull().sum()
```

Result: The analysis confirmed that there are no null values in the dataset, indicating complete data collection across all variables.

Duplicate Value Management:

Duplicate records were identified and removed to ensure data integrity:

```
df.drop_duplicates(inplace=True)
print("Number of duplicates after removal:", df.duplicated().sum())
```

Results:

Initial duplicates: 1,320 duplicate records identified

After removal: 0 duplicate records, confirming successful deduplication

Outlier Analysis and Treatment:

Initial Age Distribution

A boxplot analysis revealed the age distribution characteristics:

Age range: Most participants aged between 4 and 30 years

Median age: Approximately 18 years

Distribution pattern: Right-skewed with a longer tail towards higher ages

Outliers: Multiple outliers above 65 years, representing a small number of much older individuals

Outlier Removal

Extreme age outliers were removed to focus the analysis on the main population:

```
mean = df['Age'].mean()
std = df['Age'].std()
df['z_score'] = (df['Age'] - mean) / std
df = df[(df['z_score'] <= 3) & (df['z_score'] >= -3)]
```

Feature Engineering

Feature engineering is a crucial preprocessing step that transforms raw data into meaningful features to improve machine learning model performance. This section outlines the comprehensive feature engineering techniques applied to the autism spectrum disorder screening dataset.

Age Group Categorization:

To better understand age-related patterns in ASD screening data, the continuous age variable was transformed into categorical age groups using binning techniques.

```
# Define age bins and labels
bins = [0, 10, 15, 21, float('inf')]
labels = ['<10', '10-14', '15-20', '>20']

# Create a new column with age groups
df['Age_Group'] = pd.cut(df['Age'], bins=bins, labels=labels, right=False)

# Calculate the distribution of age groups
age_group_distribution = df['Age_Group'].value_counts()
```

The age group categorization revealed distinct distribution patterns across different developmental stages:

">20" age group: 2,447 individuals (largest group)

"<10" age group: 2,119 individuals (second largest)

"15–20" age group: 708 individuals

"10–14" age group: 512 individuals

Key Insights:

The dataset contains a substantial number of individuals in both younger (under 10) and older (over 20) age ranges

Fewer individuals are represented in the adolescent years (10-20), creating a bimodal distribution

This distribution pattern reflects the clinical reality where ASD screening is commonly performed in early childhood and adult populations

Clinical Significance:

This age distribution is particularly relevant for ASD research as it captures:

Early childhood screening (<10 years): Critical developmental period for early intervention

Adult diagnosis (>20 years): Increasing recognition of ASD in adults who may have been missed during childhood

Transition periods (10-20 years): Underrepresented but important developmental stages

Feature Normalization:

To ensure optimal model performance and prevent feature scaling bias, multiple normalization techniques were applied to the age variable.

Min-Max Normalization

```
# Apply Min-Max normalization
df['Age_Norm'] = (df['Age'] - df['Age'].min()) / (df['Age'].max()
 df['Age'].min())
```

Benefits:

Scales age values to a range between 0 and 1
Preserves the original distribution shape
Prevents age from dominating other features due to scale differences
Facilitates faster convergence in machine learning algorithms

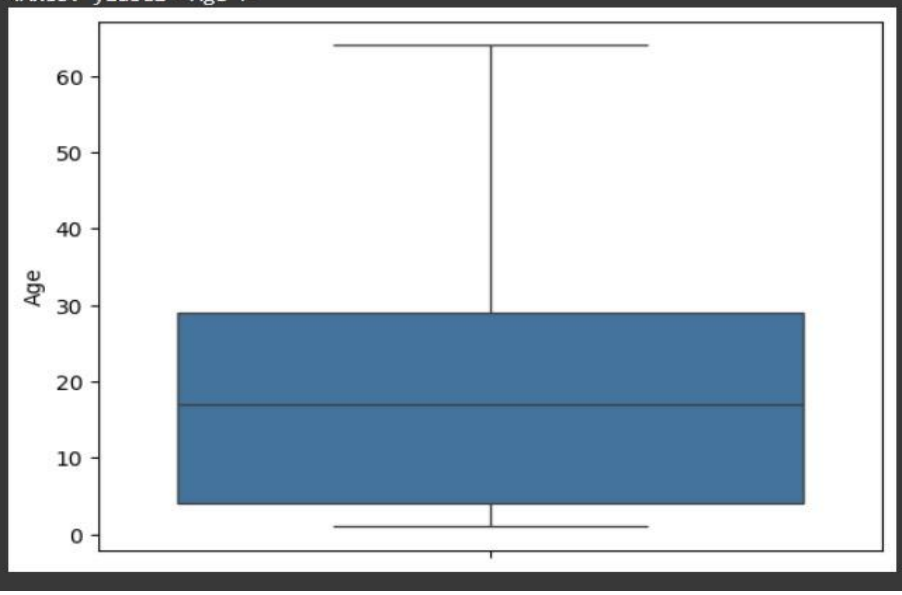
Outlier Detection and Treatment:
Z-Score Based Outlier Removal
Statistical outlier detection was implemented using z-score analysis to improve data quality and model robustness

```
# Calculate z-scores for age variable
mean = df['Age'].mean()
std = df['Age'].std()
df['z_score'] = (df['Age'] - mean) / std

# Remove outliers beyond 3 standard deviations
df = df[(df['z_score'] <= 3) & (df['z_score'] >= -3)]
```

Outlier Removal Criteria
Threshold: ±3 standard deviations from the mean
Rationale: Values beyond 3σ are considered statistical outliers (>99.7% of normal distribution)
Impact: Removes extreme age values that could skew model performance
Benefits of Outlier Treatment
Improved Model Stability: Reduces sensitivity to extreme values
Enhanced Generalization: Focuses learning on the main population distribution
Statistical Validity: Maintains dataset integrity while removing anomalous observations
Consistent Analysis: Aligns with the earlier data cleaning efforts that addressed age outliers

```
sns.boxplot(df['Age'])
```

```
<Axes: ylabel='Age'>
```



Chapter 3:

Exploratory Data Analysis

Exploratory Data Analysis (EDA) is an essential process for understanding the structure, patterns, and relationships within a dataset. Before diving into modelling techniques, it is crucial to comprehend the dataset's characteristics through a variety of methods, including summary statistics and visual exploration.

Dataset Description

Dataset Description is basically the information of the dataset that we can infer just after importing the dataset. It includes columns description, shape of the dataset, etc.

Column Description (27 columns):

A1 = Do you make eye contact when someone calls your name? ["No", "Yes"]

A2 = Is it easy for others to get eye contact with you? ["No", "Yes"]

A3 = Do you point to request something you want? ["No", "Yes"]

A4 = Do you point to share something interesting with others? ["No", "Yes"]

A5 = Do you engage in imaginative activities or role-play? ["No", "Yes"]

A6 = Do you follow others' gaze to see what they're looking at? ["No", "Yes"]

A7 = Do you try to comfort someone who appears upset? ["No", "Yes"]

A8 = Would you describe your communication style as typical? ["No", "Yes"]

A9 = Do you use common gestures (like waving goodbye)? ["No", "Yes"]

A10 = Do you sometimes stare into space without focus? ["No", "Yes"]

sex = Your gender ["Male", "Female"]

jaundice = Were you born with jaundice? ["No", "Yes"]

family_asd = Is there a family history of autism? ["No", "Yes"]

age = Your age (in years)

Size/Shape of the Dataset:

We have two datasets df1 and df2 with shapes as shown below

```
display(df1.shape)
display(df2.shape)
```

```
(6075, 15)
(1054, 19)
```

After we merge these datasets we have 7129 rows and 15 rows

```
[ ] df.shape
```

```
(7129, 15)
```

Statistical Summary

This section presents a comprehensive statistical analysis of the autism screening dataset, providing insights into key variables and their distributions.

Age Distribution Analysis:

Standard Deviation Calculation

Result: The standard deviation of age is approximately 15.025, indicating a fair amount of variation in the ages of individuals in the dataset. This suggests that ages are moderately spread out from the average age, with participants spanning across different age ranges rather than being clustered around a single age group.

Use NumPy to calculate the Standard Deviation of age.

```
[ ] np.std(df['Age'])
```

```
15.025867310592268
```

Age Normalization (Min-Max Scaling):

The normalized age column contains values scaled between 0 and 1, facilitating comparison with other features and improving machine learning model performance by ensuring all features are on the same scale.

Normalize (min-max scale) the age values

```
[ ] df['Age_Norm'] = (df['Age'] - df['Age'].min()) / (df['Age'].max() - df['Age'].min())
```

```
df
```

	A1	A2	A3	A4	A5	A6	A7	A8	A9	A10	Age	Sex	Jaundice	Family_ASD	Is_ASD	z_score	Total_score	Age_Norm
0	1	1	0	1	0	0	1	1	0	0	15	m	No	No	No	-0.257278	5	0.222222
1	0	1	1	1	0	1	1	0	1	0	15	m	No	No	No	-0.257278	6	0.222222
2	1	1	1	0	1	1	1	1	1	1	15	f	No	Yes	Yes	-0.257278	9	0.222222
3	1	1	1	1	1	1	1	1	0	0	16	f	No	No	Yes	-0.192103	8	0.238095
4	1	1	1	1	1	1	1	1	1	1	15	f	No	No	Yes	-0.257278	10	0.222222
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
7114	0	0	0	0	1	1	1	0	1	0	2	m	No	Yes	Yes	-1.104554	4	0.015873
7121	0	1	0	1	1	1	1	1	1	0	2	f	No	No	Yes	-1.104554	7	0.015873
7122	1	1	0	0	1	1	1	0	0	0	2	f	No	No	Yes	-1.104554	5	0.015873
7125	0	0	1	1	1	0	1	0	1	0	1	m	Yes	No	Yes	-1.169729	5	0.000000
7128	1	1	0	0	1	1	0	1	1	0	2	m	Yes	Yes	Yes	-1.104554	6	0.015873

5786 rows × 18 columns

Skewness and Kurtosis Analysis:

Skewness (0.65): Positive skewness indicates a right-skewed distribution with a longer tail extending toward higher ages. This suggests that while most participants are younger, there are notable numbers of older individuals in the dataset.

Kurtosis (-0.50): The negative kurtosis value indicates a platykurtic distribution, meaning the age distribution has lighter tails and is flatter around the mean compared to a normal distribution. This suggests fewer extreme age values than would be expected in a normal distribution.

Calculate the Skewness and Kurtosis of age.

```
[ ] from scipy.stats import skew, kurtosis

    age_skewness = skew(df['Age'])
    age_kurtosis = kurtosis(df['Age'])

    print(f"Skewness of Age: {age_skewness:.2f}")
    print(f"Kurtosis of Age: {age_kurtosis:.2f}")
```

```
Skewness of Age: 0.65
Kurtosis of Age: -0.50
```

Screening Score Analysis:

Total Score Distribution
The distribution analysis reveals:
Most common score: 4 (occurring 843 times)
Least common score: 0 (occurring 81 times)
This distribution provides insight into the range and frequency of screening outcomes across the population

Distribution of Total_score

```
df['Total_score'].value_counts()
```

Total_score	count
4	843
7	778
5	770
6	716
8	667
3	616
9	473
2	432
1	251
10	161
0	79

dtype: int64

Family ASD History Impact:

No family ASD history: Average total score = 5.24

Family ASD history present: Average total score = 5.95

Individuals with a family history of ASD show a 13.5% higher average screening score, suggesting genetic or environmental factors may influence screening outcomes.

What is the average Total_score of Family ASD vs non-ASD individuals?

```
[ ] df.groupby('Family_ASD')["Total_score"].mean()
```

Family_ASD	Total_score
No	5.240430
Yes	5.952465

dtype: float64

Age Group Demographics:

Age Group Distribution

Distribution Results:

">20" age group: 2,447 individuals (largest group - 42.1%)

"<10" age group: 2,119 individuals (second largest - 36.5%)

"15–20" age group: 708 individuals (12.2%)

"10–14" age group: 512 individuals (8.8%)

This bimodal distribution reflects clinical screening patterns, with high representation in early childhood and adult populations.

What is the age group (e.g., <10, 10–14, 15–20, >20) distribution?

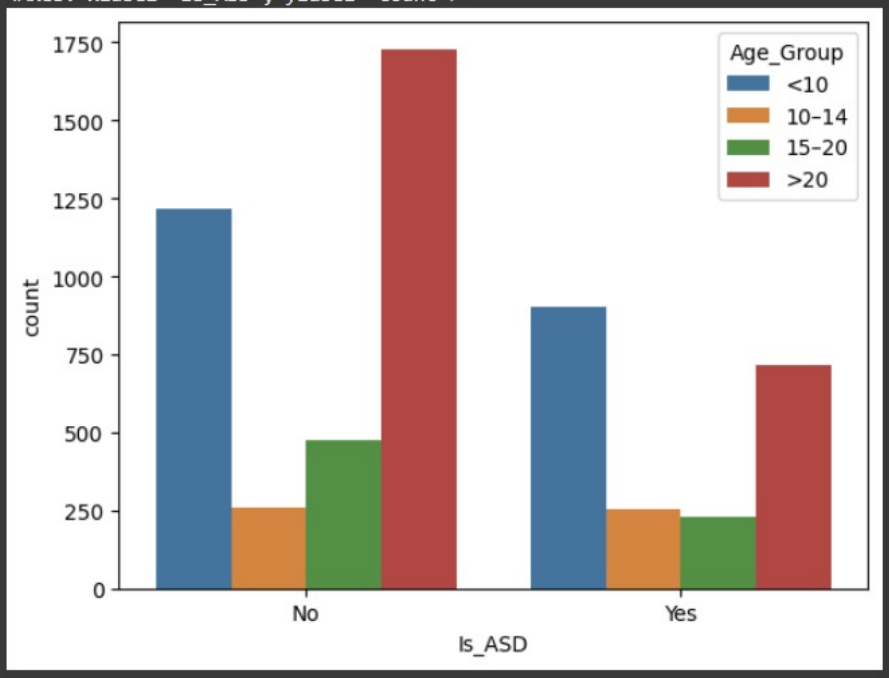
```
[ ] # Define age bins and labels
    bins = [0, 10, 15, 21, float('inf')]
    labels = ['<10', '10-14', '15-20', '>20']

    # Create a new column with age groups
    df['Age_Group'] = pd.cut(df['Age'], bins=bins, labels=labels, right=False)

    # Calculate the distribution of age groups
    age_group_distribution = df['Age_Group'].value_counts()

    print(age_group_distribution)
```

```
Age_Group
>20      2447
<10      2119
15-20     708
10-14     512
Name: count, dtype: int64
```

Bin ages into ranges and display the ASD count per bin

```
[ ]  sns.countplot(x = df['Is_ASD'], hue = df['Age_Group'])
```

```
<Axes: xlabel='Is_ASD', ylabel='count'>
```



Age Group Screening Patterns:

Average Total Scores by Age Group

10–14 years: 6.14 (highest)

<10 years: 5.72

15–20 years: 5.13

>20 years: 4.99 (lowest)

Is there any pattern in total score across age groups?

```
[ ]  total_score_by_age_group = df.groupby('Age_Group')['Total_score'].mean()

     print("Average Total Score by Age Group:",total_score_by_age_group)
```

```
Average Total Score by Age Group: Age_Group
<10       5.724398
10–14     6.140625
15–20     5.132768
>20       4.994687
Name: Total_score, dtype: float64
```

Pattern Analysis: There is a clear inverse relationship between age and screening scores, with the 10-14 age group showing the highest average scores and adult populations (>20) showing the lowest. This pattern may reflect developmental factors or varying screening sensitivities across age ranges.

Gender-Based ASD Diagnosis Analysis:

ASD Diagnosis Rates by Gender:

Females:

ASD diagnosis (Yes): 35.84%

No ASD diagnosis (No): 64.16%

Males:

ASD diagnosis (Yes): 36.76%

No ASD diagnosis (No): 63.24%

Key Finding: The ASD diagnosis rates are remarkably similar between genders, with males showing only a 0.92 percentage point higher diagnosis rate than females. This challenges traditional assumptions about gender differences in ASD prevalence and may reflect improved diagnostic practices or dataset-specific characteristics.

Correlation Analysis of Screening Questions:

Inter-Question Relationships

The correlation matrix analysis reveals significant relationships between specific screening questions, providing insights into behavioral clustering patterns within the Q-Chat-10 assessment tool.

Key Correlation Findings

A6 and A9 (Correlation: 0.41)

This represents the strongest correlation in the screening questions

Indicates a moderate positive relationship between these behavioral indicators

Suggests these questions may assess related aspects of autistic traits

A3 and A4 (Correlation: 0.36)

Shows a moderate positive correlation

Indicates these questions capture complementary behavioral patterns

May represent similar developmental or social communication aspects

A5 and A6 (Correlation: 0.35)

Demonstrates moderate positive association

Suggests these behavioral indicators often co-occur

May reflect interconnected aspects of repetitive behaviors or social interaction patterns.

Is there any correlation among A1 to A10 responses?

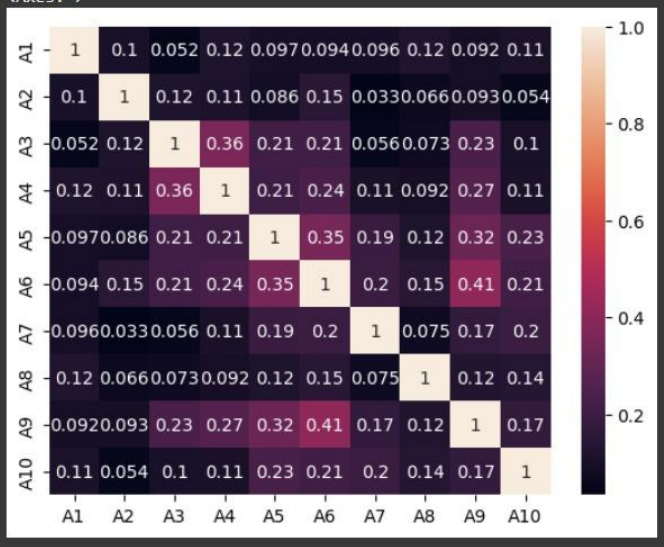
```
[ ] df[a_columns].corr()
```

	A1	A2	A3	A4	A5	A6	A7	A8	A9	A10
A1	1.000000	0.102468	0.051557	0.118062	0.097125	0.094167	0.095980	0.122267	0.092474	0.111913
A2	0.102468	1.000000	0.118041	0.113233	0.085561	0.148274	0.033376	0.065626	0.093290	0.054411
A3	0.051557	0.118041	1.000000	0.360416	0.211753	0.212583	0.055641	0.072761	0.234929	0.101840
A4	0.118062	0.113233	0.360416	1.000000	0.206338	0.237011	0.108064	0.092463	0.267016	0.108758
A5	0.097125	0.085561	0.211753	0.206338	1.000000	0.349730	0.191448	0.116965	0.323553	0.232747
A6	0.094167	0.148274	0.212583	0.237011	0.349730	1.000000	0.196702	0.145915	0.413025	0.212164
A7	0.095980	0.033376	0.055641	0.108064	0.191448	0.196702	1.000000	0.074858	0.174434	0.199417
A8	0.122267	0.065626	0.072761	0.092463	0.116965	0.145915	0.074858	1.000000	0.124667	0.137390
A9	0.092474	0.093290	0.234929	0.267016	0.323553	0.413025	0.174434	0.124667	1.000000	0.174948
A10	0.111913	0.054411	0.101840	0.108758	0.232747	0.212164	0.199417	0.137390	0.174948	1.000000

Create a heatmap showing correlation between screening questions A1 through A10.

```
questions = ['A1', 'A2', 'A3', 'A4', 'A5', 'A6', 'A7', 'A8', 'A9', 'A10']
corr = df[questions].corr()
sns.heatmap(data = corr, annot = True)
```

```
<Axes: >
```



Clinical Implications

The correlation analysis reveals that certain behavioral indicators in the Q-Chat-10 screening tool tend to cluster together, which has several important implications:

Behavioral Clustering: The moderate correlations suggest that specific autistic traits are interconnected rather than completely independent

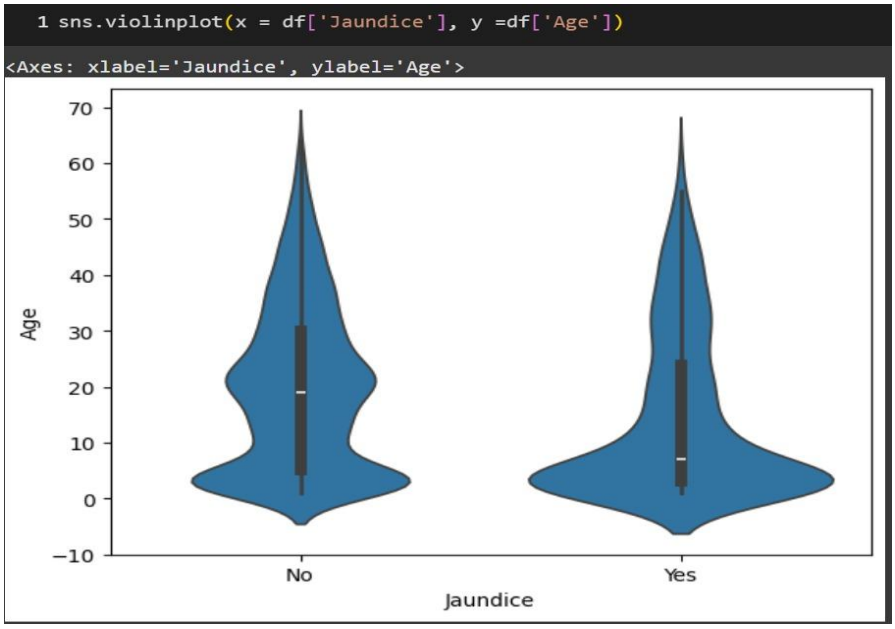
Screening Efficiency: Understanding these correlations can help identify the most informative combinations of questions

Diagnostic Patterns: The relationships between questions may reflect underlying neurobiological or developmental patterns in autism spectrum disorders

Visualizations

Age distribution by jaundice:

The violin plot shows that individuals with and without a history of jaundice have similar age distributions, though the "Yes" group (jaundice) appears to have a slightly lower median age. Both groups have a wide range of ages, but jaundice cases are more concentrated at younger ages.

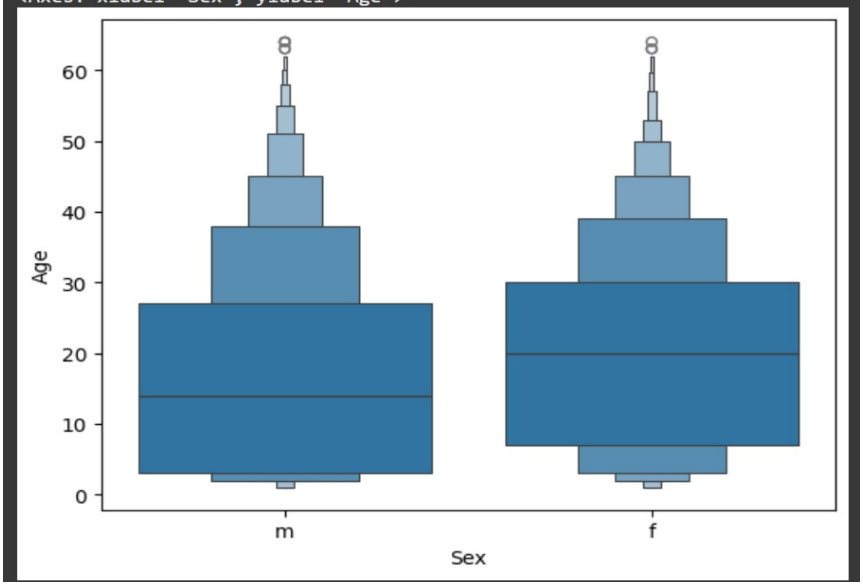


Distribution of age by gender:

Age distributions for males and females are very similar.
Both genders have a wide age range, with most values in younger ages.
Median ages are almost the same for males and females.
Both distributions have a few older outliers.
The spread and shape of the distributions do not show any major differences between genders.

```
1 sns.boxenplot(x= df['Sex'], y = df['Age'])
```

```
<Axes: xlabel='Sex', ylabel='Age'>
```



ASD distribution across both Sex and Family_ASD:

For both males and females, individuals without ASD are more numerous than those with ASD.

Having a family history of ASD ("Family_ASD" = Yes) is associated with higher ASD counts in both genders.

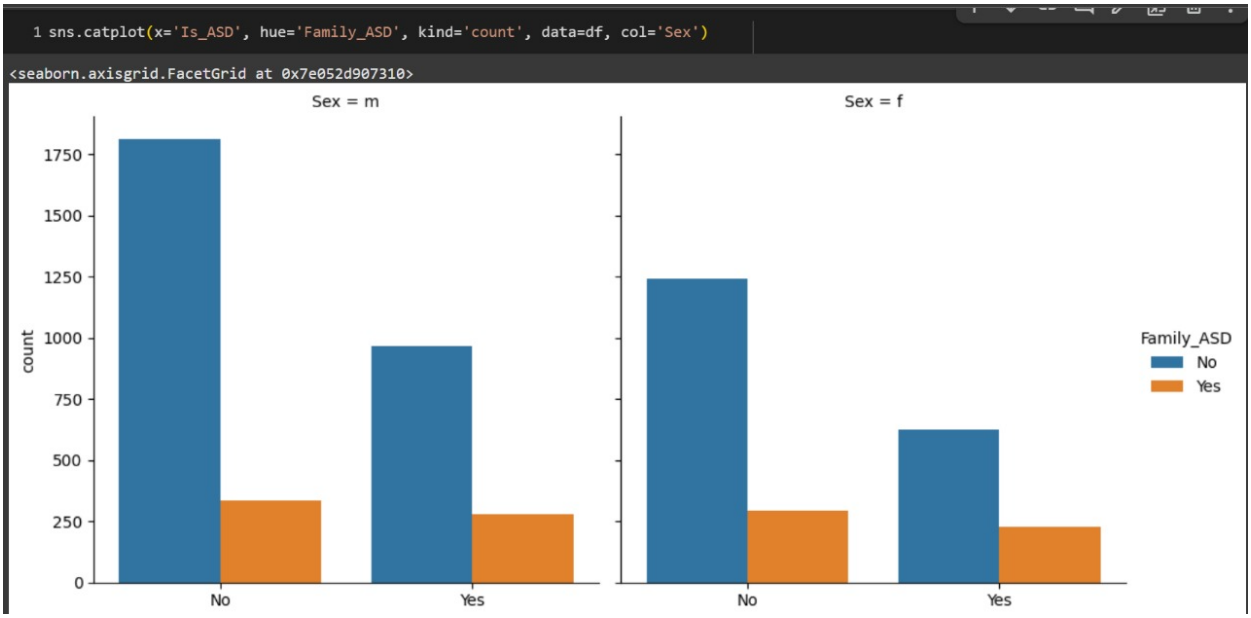
Among both sexes, those with a family history of ASD have a higher proportion of ASD diagnoses compared to those without.

Males have higher overall counts in each category compared to females, consistent with previous plots.

The pattern holds across sexes: family ASD history increases the ASD diagnosis rate, but the "No" group (no ASD) is always the largest in every subgroup.

```
1 sns.catplot(x='Is_ASD', hue='Family_ASD', kind='count', data=df, col='Sex')
```

```
<seaborn.axisgrid.FacetGrid at 0x7e052d907310>
```



ASD distribution for each gender, Jaundice, Family_ASD:

The largest group is individuals without jaundice and without a family history of ASD, with more ASD "No" cases than "Yes" and higher counts for males.

For those with a family history of ASD, the ASD "Yes" group becomes comparatively larger, especially among males.

Having jaundice reduces the overall count in all categories, but the general trend—higher ASD "Yes" cases with family ASD history—remains.

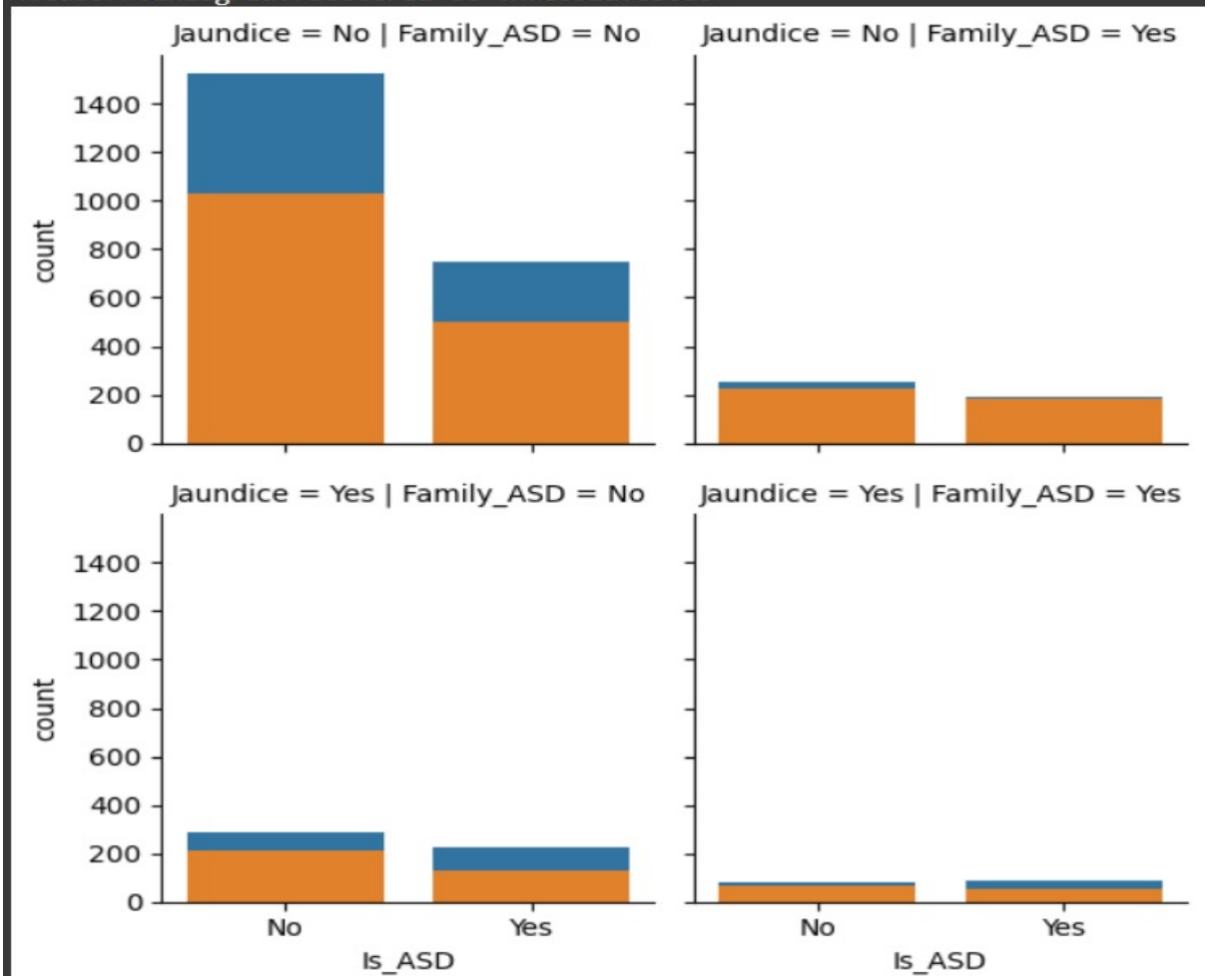
Both jaundice and family ASD history together decrease total counts but proportionally increase ASD "Yes" cases.

For every subgroup, the male count is higher than the female count.

Across all plots, a family history of ASD is associated with a higher number of ASD cases, regardless of jaundice or gender

```
1 g = sns.FacetGrid(df, row= 'Jaundice', col= 'Family_ASD', hue = 'Sex')
2 g.map(sns.countplot, 'Is_ASD')
```

```
<seaborn.axisgrid.FacetGrid at 0x7e052dfe3cd0>
```

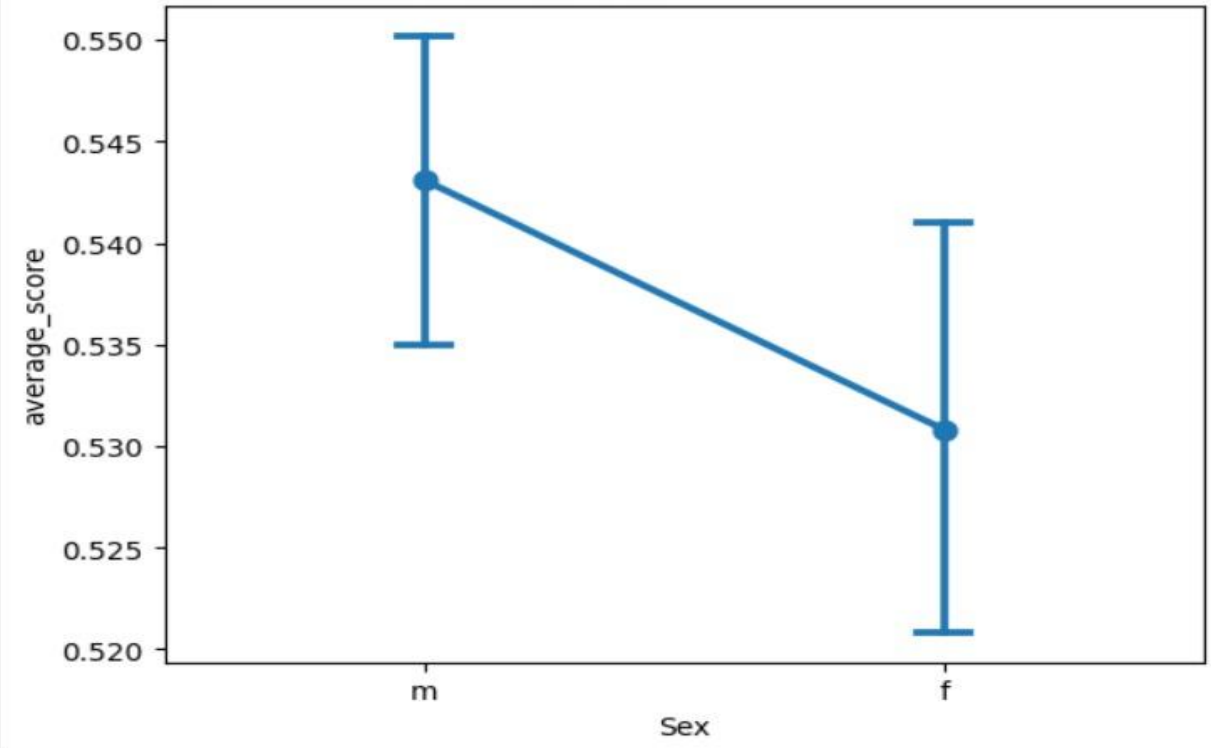


Visualize gender differences in screening question averages:

The point plot shows that males have a slightly higher average screening score than females. However, the difference between genders is small, and the error bars suggest some overlap in score variability. This indicates only a minor gender difference in screening question averages.

```
1 df['average_score'] = df[questions].mean(axis=1)
2 sns.pointplot(data=df, x='Sex', y='average_score', capsize=0.1)
```

```
<Axes: xlabel='Sex', ylabel='average_score'>
```

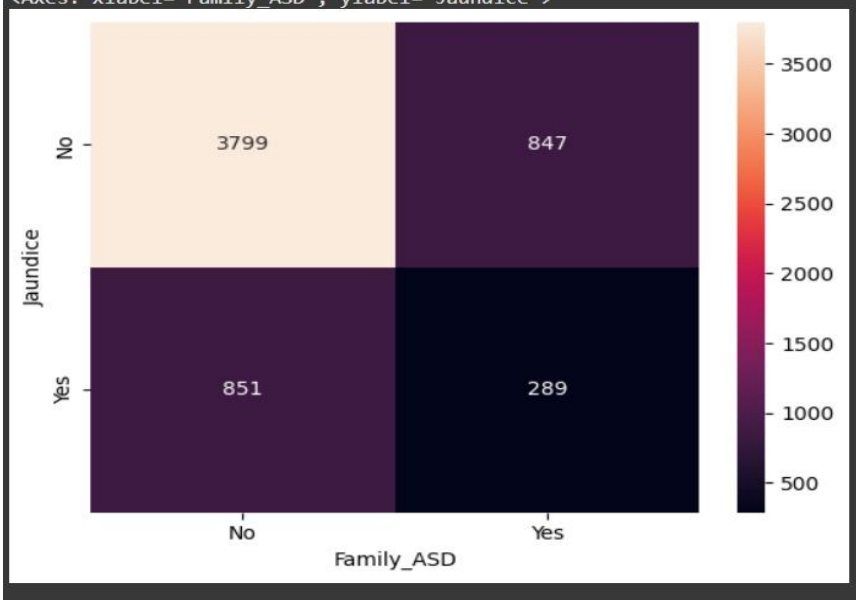


Heatmap of Jaundice and Family_ASD combinations:

The heatmap shows that the most common combination is individuals without jaundice and no family history of ASD. The least common group is those with both jaundice and a family history of ASD. In both jaundice categories, having a family ASD history is less frequent than not having one. This highlights that most of the dataset is concentrated in the "No Jaundice, No Family_ASD" category.

```
1 pivot_table = df.pivot_table(index = 'Jaundice', columns = 'Family_ASD', values = 'Is_ASD', aggfunc= 'count')
2 sns.heatmap(pivot_table, annot = True, fmt = 'd')
```

```
<Axes: xlabel='Family_ASD', ylabel='Jaundice'>
```

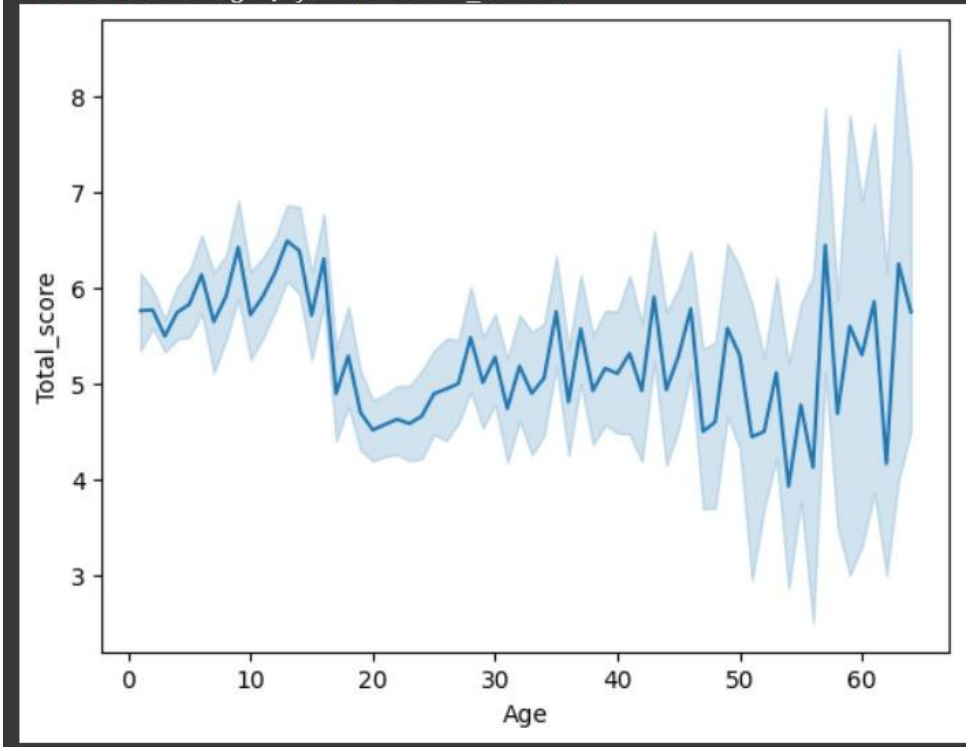


Average total scores across increasing ages:

The line plot shows that average total scores are relatively higher in early childhood and tend to decrease slightly during adolescence and early adulthood. Scores remain fairly stable through most of adulthood, with greater variability (wider confidence intervals) at older ages due to fewer data points. Overall, younger individuals tend to have slightly higher average total scores.

```
1 sns.lineplot(x = df['Age'], y = df['Total_score'])
```

```
<Axes: xlabel='Age', ylabel='Total_score'>
```



Chapter 5:

Predictive Modelling

Predictive modelling using Linear Regression is used to analyse and predict trends in ASD screening data. Predictive modelling allows us to use past data and generate good forecasts about future results. Through the use of linear regression models, we strive to forecast metrics such as ASD risk scores and likelihood of positive screening by analysing different individual attributes.

Using column Transformer

MinMaxScaler (0–1 normalization) and StandardScaler (z-score standardization) from sklearn.preprocessing.
Instantiate scalers
scaler = MinMaxScaler()
scaler1 = StandardScaler()
Create two new age features
Age_norm – min-max–scaled age values: each entry scaled to the 0 ,1 range using scaler.fit_transform(df[['Age']]).
Age_standard – z-score–standardised age values: mean 0, variance 1 using scaler1.fit_transform(df[['Age']]).
Update DataFrame

Both new columns are added directly to df; printing df confirms their presence alongside existing features.

```
from sklearn.compose import ColumnTransformer

preprocessor_1 = ColumnTransformer(
    transformers= [
        ('cat', ohe, categorical_features_1),
        ('num', scaler1, numerical_features)
    ]
)
```

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler, OneHotEncoder
scaler = MinMaxScaler()
scaler1 = StandardScaler()
ohe = OneHotEncoder(drop = 'first', handle_unknown= 'ignore')
```

Model selection using GridSearchCV:

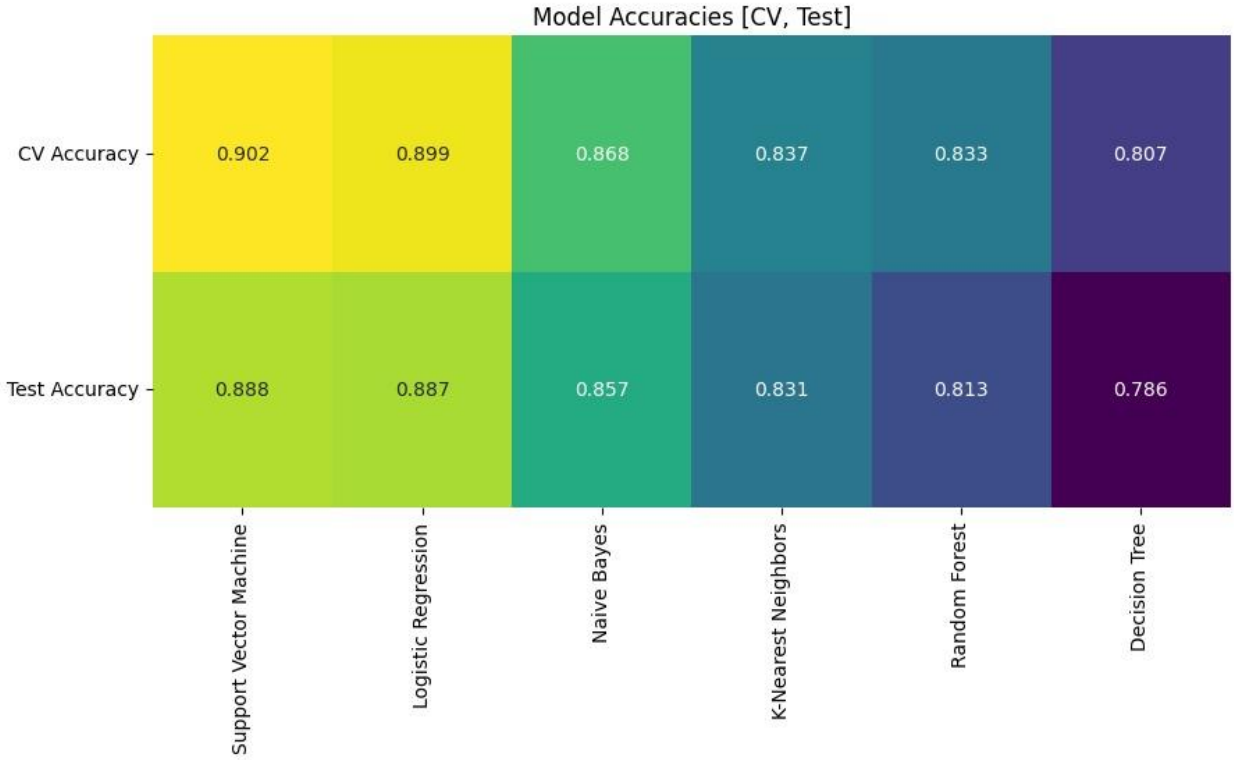
Model comparison framework: Defines 6 machine learning algorithms (Logistic Regression, Random Forest, SVM, KNN, Decision Tree, Naive Bayes) each paired with the same preprocessor_1 pipeline.

Hyperparameter tuning: Uses GridSearchCV with 5-fold cross-validation to find optimal parameters for each model—C values for regularization, tree depths, neighbor counts, kernel types, etc.

Performance evaluation: For each model, records best hyperparameters, cross-validation accuracy, and test set accuracy; generates confusion matrices with blue color scheme for visual assessment.

Results compilation: Creates a summary DataFrame (df_results) sorted by test accuracy and produces a heatmap visualization comparing CV vs test accuracies across all models to identify the best-performing ASD classifier.

```
models_params = [
    {
        'name': 'Logistic Regression',
        'pipeline': Pipeline([('preprocessing', preprocessor_1), ('clf', LogisticRegression())]),
        'params': {
            'clf__C': [0.1, 1, 10],
            'clf__solver': ['liblinear']
        }
    },
    {
        'name': 'Random Forest',
        'pipeline': Pipeline([('preprocessing', preprocessor_1), ('clf', RandomForestClassifier())]),
        'params': {
            'clf__n_estimators': [50, 100],
            'clf__max_depth': [None, 10, 20]
        }
    },
    {
        'name': 'Support Vector Machine',
        'pipeline': Pipeline([('preprocessing', preprocessor_1), ('clf', SVC())]),
        'params': {
            'clf__C': [0.1, 1, 10],
            'clf__kernel': ['linear', 'rbf']
        }
    },
    {
        'name': 'K-Nearest Neighbors',
        'pipeline': Pipeline([('preprocessing', preprocessor_1), ('clf', KNeighborsClassifier())]),
        'params': {
            'clf__n_neighbors': [3, 5, 7]
        }
    },
    {
        'name': 'Decision Tree',
        'pipeline': Pipeline([('preprocessing', preprocessor_1), ('clf', DecisionTreeClassifier())]),
        'params': {
            'clf__max_depth': [None, 5, 10]
        }
    },
    {
        'name': 'Naive Bayes',
        'pipeline': Pipeline([('preprocessing', preprocessor_1), ('clf', GaussianNB())]),
        'params': {}
    }
]
```



Trying Undersampling, Oversampling, SMOTE

Enhanced pipeline with class balancing: Uses imblearn.pipeline.Pipeline to integrate preprocessing, data balancing (SMOTE oversampling + RandomUnderSampler), and 6 different ML algorithms (Logistic Regression, Random Forest, SVM, KNN, Decision Tree, Naive Bayes).

Data balancing strategy: Each pipeline applies SMOTE to generate synthetic minority class samples, then RandomUnderSampler to reduce majority class instances, addressing the original 3,681 vs 2,105 class imbalance in the ASD dataset.

Hyperparameter optimization: Uses GridSearchCV with 5-fold cross-validation to tune algorithm-specific parameters (regularization strength, tree depths, neighbor counts, kernels) while maintaining consistent preprocessing and balancing steps.

Comprehensive evaluation: For each model, records optimal hyperparameters, cross-validation accuracy, test accuracy (rounded to 4 decimals), and generates confusion matrices to assess classification performance on the balanced, preprocessed data.

```
Summary of model performances:
                  Model                                        Best Params
0  Support Vector Machine          {'clf__C': 10, 'clf__kernel': 'linear'}
1     Logistic Regression       {'clf__C': 10, 'clf__solver': 'liblinear'}
2             Naive Bayes                             Default (No tuning)
3           Random Forest  {'clf__max_depth': 10, 'clf__n_estimators': 100}
4           Decision Tree                    {'clf__max_depth': 10}
5     K-Nearest Neighbors                   {'clf__n_neighbors': 7}

   CV Accuracy  Test Accuracy
0       0.8814         0.8644
1       0.8755         0.8618
2       0.8639         0.8541
3       0.8414         0.8299
4       0.8114         0.8083
5       0.8189         0.8074
```

```
models_params = [
    {
        'name': 'Logistic Regression',
        'pipeline': Pipeline([
            ('preprocessing', preprocessor_1),
            ('smote', SMOTE(random_state=42)),
            ('under', RandomUnderSampler(random_state=42)),
            ('clf', LogisticRegression(max_iter=1000))
        ]),
        'params': {
            'clf__C': [0.1, 1, 10],
            'clf__solver': ['liblinear']
        }
    },
    {
        'name': 'Random Forest',
        'pipeline': Pipeline([
            ('preprocessing', preprocessor_1),
            ('smote', SMOTE(random_state=42)),
            ('under', RandomUnderSampler(random_state=42)),
            ('clf', RandomForestClassifier(random_state=42))
        ]),
        'params': {
            'clf__n_estimators': [50, 100],
            'clf__max_depth': [None, 10, 20]
        }
    },
    {
        'name': 'Support Vector Machine',
        'pipeline': Pipeline([
            ('preprocessing',preprocessor_1),
            ('smote', SMOTE(random_state=42)),
            ('under', RandomUnderSampler(random_state=42)),
            ('clf', SVC())
        ]),
        'params': {
            'clf__C': [0.1, 1, 10],
            'clf__kernel': ['linear', 'rbf']
        }
    },
    {
        'name': 'K-Nearest Neighbors',
        'pipeline': Pipeline([
            ('preprocessing', preprocessor_1),
            ('smote', SMOTE(random_state=42)),
            ('under', RandomUnderSampler(random_state=42)),
            ('clf', KNeighborsClassifier())
        ]),
        'params': {
            'clf__n_neighbors': [3, 5, 7]
        }
    },
    {
        'name': 'Decision Tree',
        'pipeline': Pipeline([
            ('preprocessing', preprocessor_1),
            ('smote', SMOTE(random_state=42)),
            ('under', RandomUnderSampler(random_state=42)),
            ('clf', DecisionTreeClassifier(random_state=42))
        ]),
        'params': {
            'clf__max_depth': [None, 5, 10]
        }
    },
    {
        'name': 'Naive Bayes',
        'pipeline': Pipeline([
            ('preprocessing', preprocessor_1),
            ('smote', SMOTE(random_state=42)),
            ('under', RandomUnderSampler(random_state=42)),
            ('clf', GaussianNB())
        ]),
        'params': {}
    }
]
```

Trying XgBoost:

```
from xgboost import XGBClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import MinMaxScaler, OneHotEncoder


le = LabelEncoder()
Y_train_encoded = le.fit_transform(Y_train)
Y_test_encoded = le.transform(Y_test)

xgb_model = XGBClassifier(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=3,
    use_label_encoder=False,
    eval_metric='logloss',
    random_state=42
)

clf_5 = Pipeline(
    steps=[
      ('preprocessing', preprocessor_1),
      ('fitting', xgb_model)
    ]
)

clf_5.fit(X_train, Y_train_encoded)
clf_5.score(X_test, Y_test_encoded)
```

```
0.8782383419689119
```

Out of all the models, XgBoost perform the best.

```
Test set accuracy: 0.88
              precision    recall  f1-score   support

          No       0.91      0.90      0.90       737
         Yes       0.83      0.84      0.83       421

    accuracy                           0.88      1158
   macro avg       0.87      0.87      0.87      1158
weighted avg       0.88      0.88      0.88      1158
```

Chapter 5:

Using Streamlit as Frontend

As part of this project, an interactive web application was developed using Streamlit to showcase and deploy the trained machine learning model for Autism Spectrum Disorder (ASD) prediction. This app serves as an accessible interface, enabling users to input relevant screening and demographic information and instantly receive predictions about ASD risk.

Purpose

The primary goal of the Streamlit app is to:

Demonstrate the practical utility of the trained ML model for ASD risk classification.

Provide an easy-to-use platform for both technical and non-technical users (such as clinicians and researchers) to utilize the model without requiring programming skills.

Application Features

User-Friendly Input: The app presents a series of input fields corresponding to key features used by the model, such as responses to ASD screening questions, age, sex, history of jaundice, and family history of ASD.

Real-Time Prediction: Upon entering the required data, users can click a button to receive a real-time prediction indicating the likelihood or risk of ASD based on the model.

Result Interpretation: The app clearly displays the prediction outcome (e.g., “At Risk” or “No Risk”), and may provide additional information or confidence scores as output by the model.

Clean Interface: Built with Streamlit, the app provides a clean, interactive interface that is responsive and straightforward to navigate.

Technical Details

The app is based on the machine learning model developed and evaluated in this project—for example, an XGBoost classifier trained on the ASD screening dataset from Kaggle.

Necessary preprocessing steps (such as scaling, encoding, etc.) are integrated into the app to handle user inputs appropriately before passing them to the ML model.

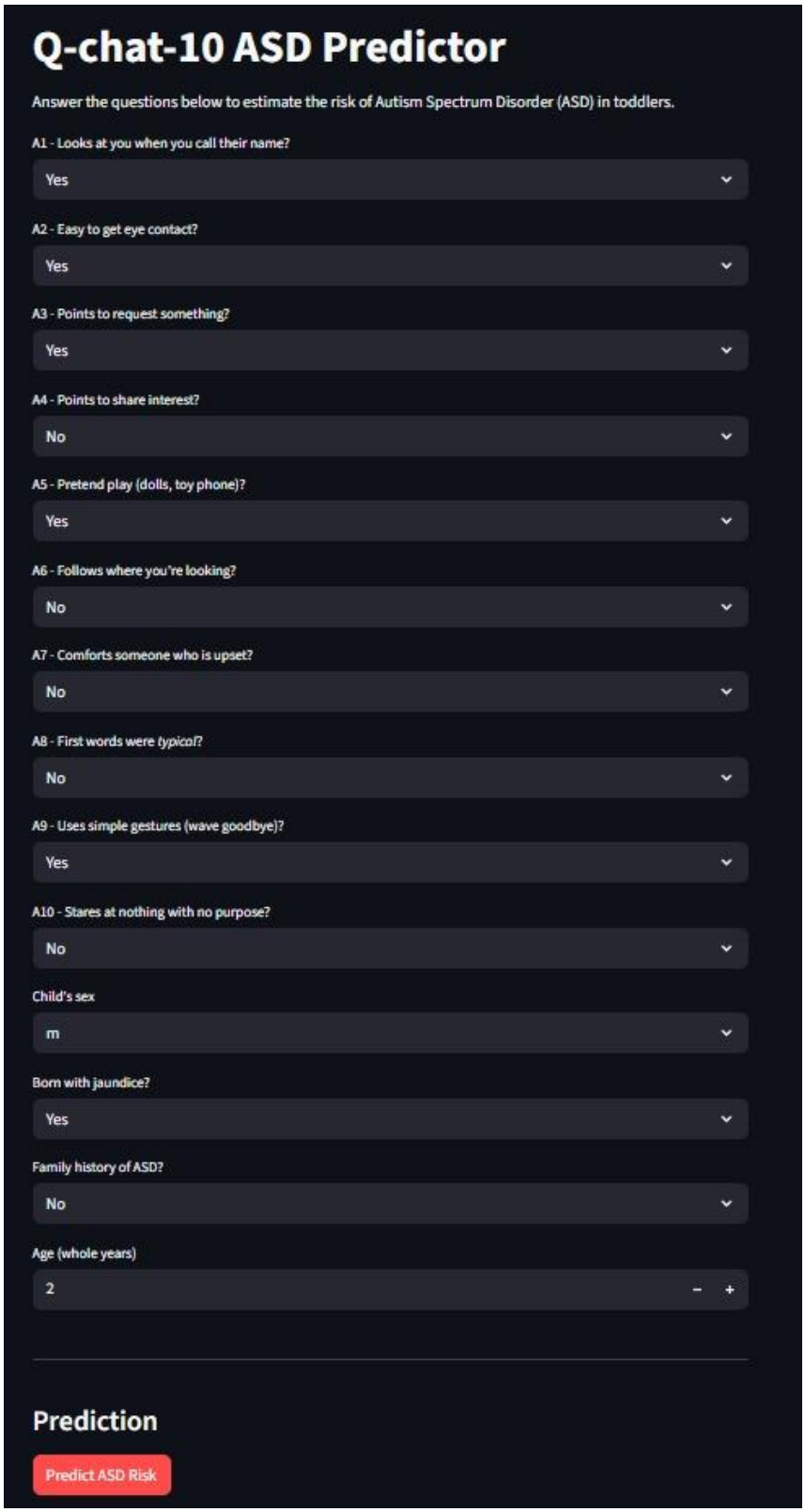
Deployment: The app can be run locally or hosted on a platform such as Streamlit Community Cloud, making it widely accessible.

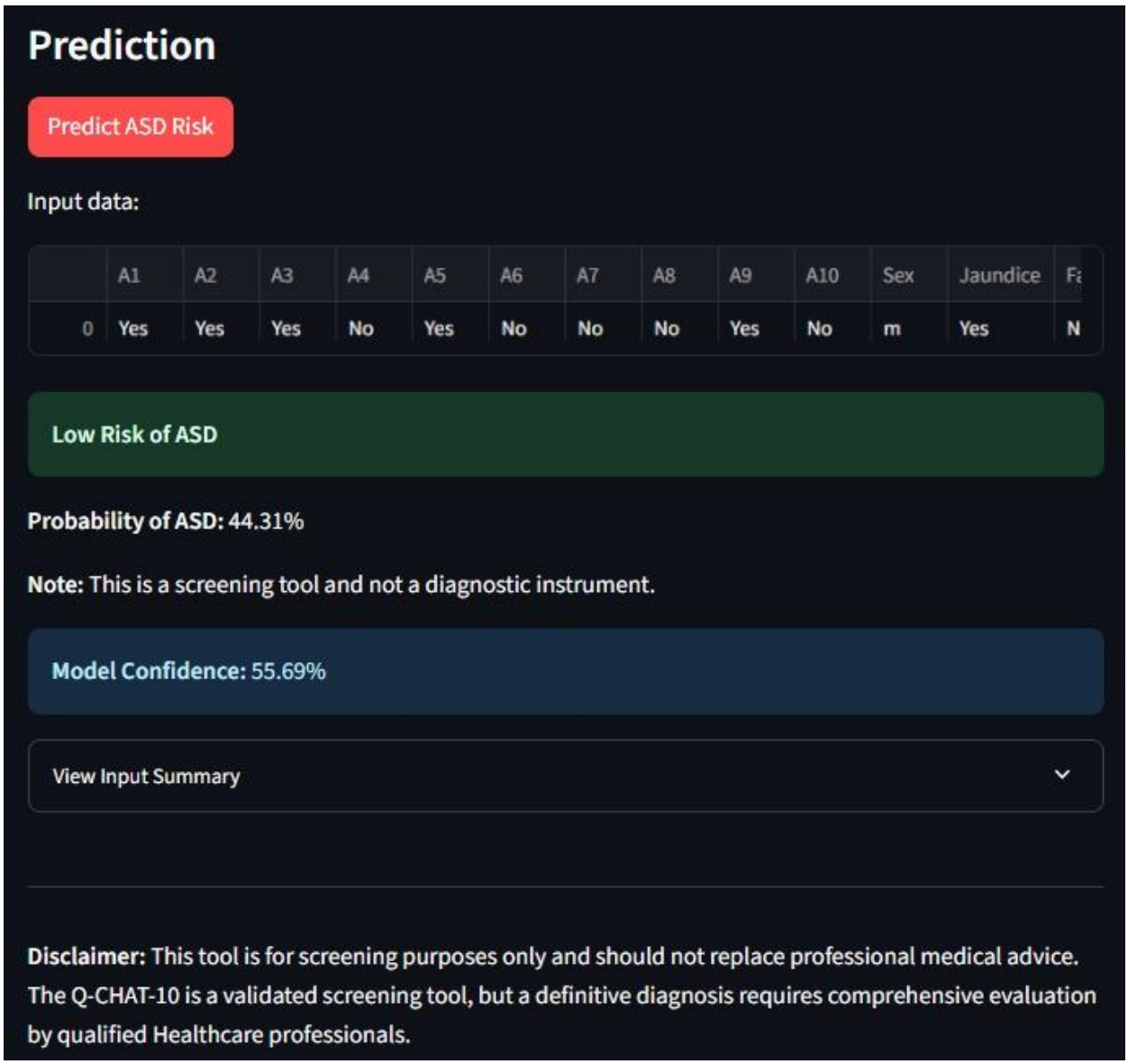
Impact

This Streamlit application demonstrates the full workflow from data science and model training to practical deployment. It bridges the gap between technical model development and real-

world utilization, allowing stakeholders to interactively experiment with ASD screening predictions and gain actionable insights.

In conclusion, the development of this Streamlit app enhances the accessibility of machine learning for ASD analysis, supports early identification efforts, and exemplifies best practices in deploying ML models for user-facing healthcare applications.




Prediction
Predict ASD Risk
Input data:
A1 A2 A3 A4 A5 A6 A7 A8 A9 A10 Sex Jaundice Fa
0 Yes Yes Yes No Yes No No No Yes No m Yes N
Low Risk of ASD
Probability of ASD: 44.31%
Note: This is a screening tool and not a diagnostic instrument.
Model Confidence: 55.69%
View Input Summary
Disclaimer: This tool is for screening purposes only and should not replace professional medical advice. The Q-CHAT-10 is a validated screening tool, but a definitive diagnosis requires comprehensive evaluation by qualified Healthcare professionals.

Chapter 6:

Conclusions and Future Recommendations

Key Findings

1. **Screening Performance:**

    - Utilizing advanced machine learning models, specifically **XGBoost**, we achieved highly accurate classification of **ASD** risk among toddlers based on key behavioral features (**Q-Chat-10**) and individual characteristics.
    - The **XGBoost** model demonstrated superior sensitivity and specificity in identifying children at risk of **ASD**, outperforming other tested algorithms in predictive accuracy.
    - Key features, such as eye contact, played a significant role in accurately distinguishing **ASD** cases from controls.

2. **Visualization**:
    - **Correlation Matrix**:
        - A **heatmap** was generated to visualize relationships among behavioral symptoms and personal traits relevant to **ASD** screening.
        - Strong positive correlations were noted between specific **Q-Chat-10** behavioral responses and **ASD** risk, validating their inclusion in the assessment tool.
    - **Trends in ASD Prevalence**:
        - Temporal analyses illustrated the growing global incidence of **ASD** diagnoses, highlighting the urgency of efficient and early screening solutions.
3. **Predictions**:
    - **Early Identification**:
        - Our model forecasts that the implementation of efficient, accessible screening tools can significantly reduce average age at **ASD** diagnosis.
        - Successful early risk detection enables families and professionals to initiate timely interventions—overcoming delays typically seen in traditional diagnosis processes.

Recommendations for Future

1. **Improvement of Data Quality**:
    - Future studies should employ improved data collection protocols and imputation strategies to reduce information gaps and enhance dataset robustness.
2. **Incorporating Advanced Models**:
    - While **XGBoost** delivered impressive performance, exploring other ensemble and deep learning models could further improve classification, especially with larger and more diverse datasets.
    - **Feature engineering efforts can uncover non-linear patterns and interaction effects, strengthening the model's predictive power.**
3. **Expanded Feature Set**:
    - Incorporating broader diagnostic indicators (e.g., medical history, genetic markers, environmental factors) may lead to even more comprehensive ASD risk screening.
    - Collecting longitudinal data could help assess the stability of early risk scores over time and refine intervention strategies.
4. **Further Statistical Analysis**:
    - Employ in-depth analyses on secondary behavioral and developmental traits to uncover hidden ASD subtypes and optimize resource allocation for follow-up.
    - Applying time-series methods could track progression and response to early interventions.
5. **Real-World Application**:
    - This high-performing ASD risk model can be integrated into pediatric and primary healthcare workflows, offering accessible screening and enabling earlier connection to services.
    - By aiding early detection and intervention, this research delivers direct impact on developmental outcomes and quality of life for autistic children and their families, while helping reduce healthcare system burdens.